



Laboratorio 1 Semana #13



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingrese a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
 - **Se utilizó el método GET, ya que la operación tiene como objetivo consultar u obtener la información de una mascota registrada en la API.**
- Indique qué parámetro necesita la solicitud.
 - **La solicitud necesita el parámetro petId(int), que corresponde al identificador de la mascota que se desea consultar**
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.
 - **La respuesta contiene los datos de la mascota asociada al ID consultado. En este caso muestra el ID 1, el nombre "Pet1", las URLs de sus fotos, sus etiquetas (tags) y su estado "available", indicando que la mascota se encuentra disponible. La API respondió con el código HTTP 200, lo que indica que la consulta se realizó correctamente.**

Evidencia: captura de Swagger mostrando la operación y su respuesta.



Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Pet1", "photoUrls": ["test1", "test2"], "tags": [], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 83 content-type: application/json date: Sat, 03 Oct 2025 00:39:47 GMT referrer-policy: no-referrer-when-downgrade server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description
------	-------------

Links

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

Parameters

No parameters

Request body **required**

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 10716,
  "name": "Luna",
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "photoUrls": [
    "https://ejemplo.com/luna.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```



Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10716,
    "name": "Luna",
    "category": {
      "id": 1,
      "name": "Perro"
    },
    "photoUrls": [
      "https://ejemplo.com/luna.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "Mascota"
      }
    ],
    "status": "available"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Cancel

Name	Description
petId required	ID of pet to return
integer(\$int64) (path)	10716

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10716' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/10716

Server response

Code Details

Code Details

200

Response body

```
{
  "id": 10716,
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "name": "Luna",
  "photoUrls": [
    "https://ejemplo.com/luna.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 162
content-type: application/json
date: Sat, 30 Oct 2025 01:09:53 GMT
referrer-policy: no-referrer-when-downgrade
server: Jetty(9.4.53.v20231009)
```

Responses



Overview GET https://petstore3.swagger.io/api/v3/pet POST https://petstore3.swagger.io/api/v3/pet

POST https://petstore3.swagger.io/api/v3/pet

Docs Params Authorization Headers Body Scripts Settings

Body Cookies Headers Test Results

```
1 {
2   "id": 18716,
3   "name": "Luna",
4   "category": {
5     "id": 1,
6     "name": "Perro"
7   },
8   "photoUrls": [
9     "https://ejemplo.com/luna.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "available"
18 }
```

200 OK · 148 ms · 524 B

Overview GET https://petstore3.swagger.io/api/v3/pet/18716 POST https://petstore3.swagger.io/api/v3/pet/18716

GET https://petstore3.swagger.io/api/v3/pet/18716

Docs Params Authorization Headers Body Scripts Settings

Body Cookies Headers Test Results

```
1 {
2   "id": 18716,
3   "name": "Luna",
4   "category": {
5     "id": 1,
6     "name": "Perro"
7   },
8   "photoUrls": [
9     "https://ejemplo.com/luna.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "available"
18 }
```

200 OK · 524 ms · 524 B



4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
 - Se utilizó el método PUT, ya que permite actualizar la información de un recurso existente. En este caso se utilizó para modificar los datos de la mascota creada anteriormente.
- ¿Qué datos fueron enviados en el body?
 - Se enviaron los datos de la mascota en formato JSON, manteniendo su mismo ID y modificando principalmente el nombre y el estado.
- ¿Qué código HTTP devolvió la API?
 - La API devolvió el código HTTP 200 (OK), lo que indica que la solicitud PUT se procesó correctamente y que los datos de la mascota fueron actualizados.
- ¿Cómo comprobó que el cambio realmente ocurrió?
 - Se utilizó nuevamente el método GET con el id de la mascota y se verificaron los cambios

PUT /pet Update an existing pet

Update an existing pet by id.

Parameters Cancel Reset

No parameters

Request body required application/json

Update an existent pet in the store

Edit Value | Schema

```
{
  "id": 10716,
  "name": "Hala",
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "photoUrls": [
    "https://example.com/luna.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "No disponible"
}
```

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 10716,
    "name": "Hala",
    "category": {
      "id": 1,
      "name": "Perro"
    },
    "photoUrls": [
      "https://example.com/luna.jpg"
    ],
    "tags": [
      {
        "id": 1,
        "name": "Mascota"
      }
    ],
    "status": "No disponible"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet



```
Code Details

200

Response body

{
  "id": 10716,
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "name": "Nala",
  "photoUrls": [
    "https://example.com/luna.jpg"
  ],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "No disponible"
}

Response headers

access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 266
content-type: application/json
date: Sat, 03 Oct 2026 01:15:35 GMT
referrer-policy: no-referrer-when-downgrade
server: Jetty(9.4.53.v20231009)
```

Overview GET https://petstore3.swagger.io/api/v3/pet PUT https://petstore3.swagger.io/api/v3/pet

PUT https://petstore3.swagger.io/api/v3/pet

Docs Params Authorization Headers Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
6 {
7   "name": "Perro"
8 },
9   "photoUrls": [
10    "https://example.com/luna.jpg"
11 ],
12   "tags": [
13     {
14       "id": 1,
15       "name": "Mascota"
16     }
17 ],
18   "status": "No disponible"
19 }
```

Body Cookies Headers Test Results

JSON Preview Visualize

```
1 {
2   "id": 10716,
3   "category": {
4     "id": 1,
5     "name": "Perro"
6   },
7   "name": "Nala",
8   "photoUrls": [
9     "https://example.com/luna.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "No disponible"
18 }
```

200 OK · 797 ms · 528 B

Comprobación de que los cambios aplicados en el método PUT funcionan

GET https://petstore3.swagger.io/api/v3/pet/10716

Docs Params Authorization Headers Body Scripts Settings

Document this request... Write with AI

Body Cookies Headers Test Results

JSON Preview Visualize

```
1 {
2   "id": 10716,
3   "category": {
4     "id": 1,
5     "name": "Perro"
6   },
7   "name": "Nala",
8   "photoUrls": [
9     "https://example.com/luna.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "No disponible"
18 }
```

200 OK · 639 ms · 528 B



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
 - Se utilizó el parámetro status, asignándole inicialmente el valor available. Este parámetro permite filtrar las mascotas según su estado.
- Explique dónde se envía ese parámetro.
 - El parámetro se envía en la URL de la solicitud como un query parameter. Por ejemplo, la solicitud tendrá una estructura similar a:
GET /pet/findByStatus?status=available
- Observe la estructura de la respuesta.
 - La respuesta se presenta en formato JSON como un arreglo o lista de mascotas. Cada mascota contiene diferentes datos, como su id, nombre, categoría, fotografías, etiquetas y estado.
- Indique qué sucede si cambia el valor del parámetro por pending.
 - Al cambiar el parámetro de available a pending, la API modifica el filtro de búsqueda y devuelve únicamente las mascotas cuyo estado es pending. La estructura de la respuesta se mantiene igual; lo que cambia son los registros obtenidos según el estado seleccionado.

The screenshot shows the Swagger UI for the endpoint `GET /pet/findByStatus`. The **Parameters** section shows a required query parameter `status` with a dropdown menu set to `available`. Below this are `Execute` and `Clear` buttons. The **Responses** section shows the `200` response with a `Response body` containing a JSON array of two dog objects. The first object has `id: 12345`, `category: {id: 1, name: 'dogs'}`, `name: 'Fluffy'`, `photoUrls: []`, and `tags: [{id: 0, name: 'tag1'}, {id: 1, name: 'tag2'}]`. The second object has `id: 123456` and `category: {id: 1, name: 'Dogs'}`. Both have `status: 'available'`.

Parameters

Name	Description
<code>status</code> * required	Status values that need to be considered for filter
<code>string</code> (query)	available

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

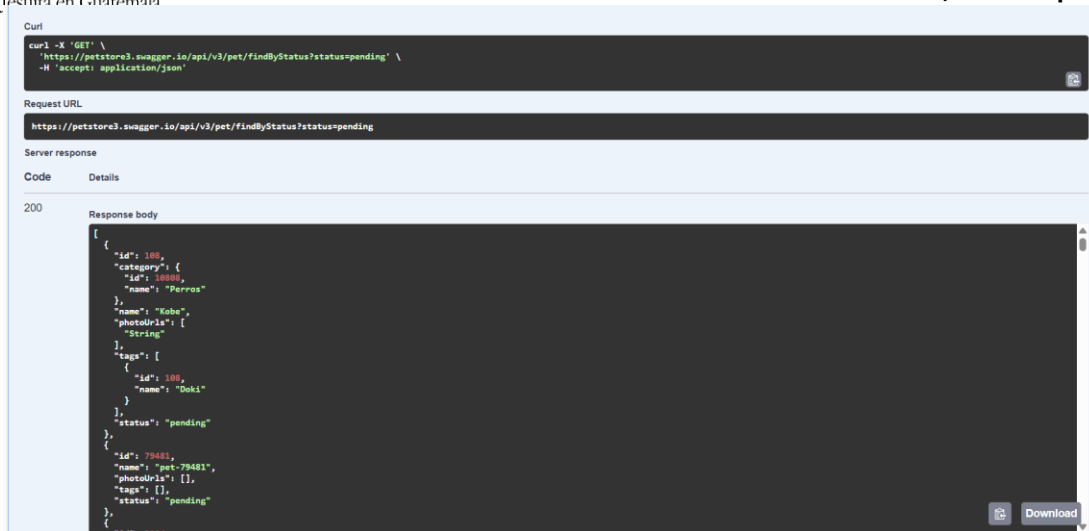
Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available
```

Server response

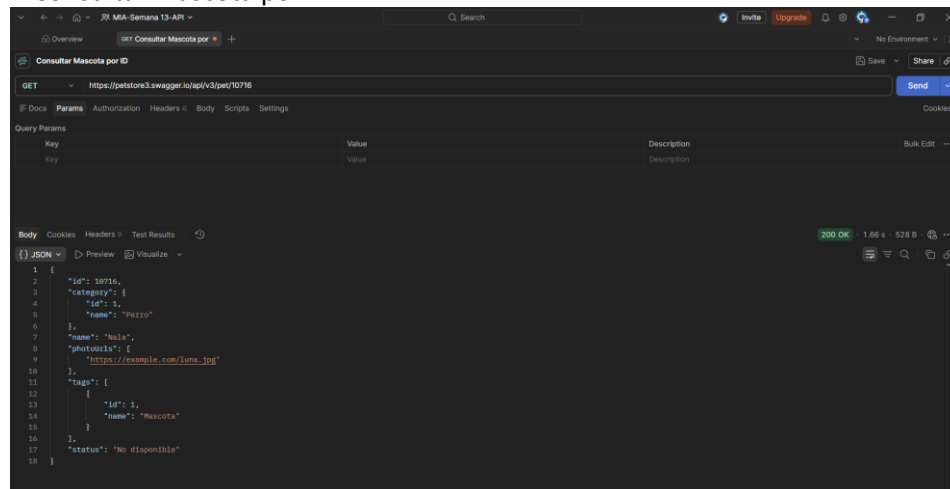
Code	Details
200	Response body <pre>{ "id": 12345, "category": { "id": 1, "name": "dogs" }, "name": "Fluffy", "photoUrls": [], "tags": [{ "id": 0, "name": "tag1" }, { "id": 1, "name": "tag2" }], "status": "available" }, { "id": 123456, "category": { "id": 1, "name": "Dogs" }, "status": "available" }</pre>

Download

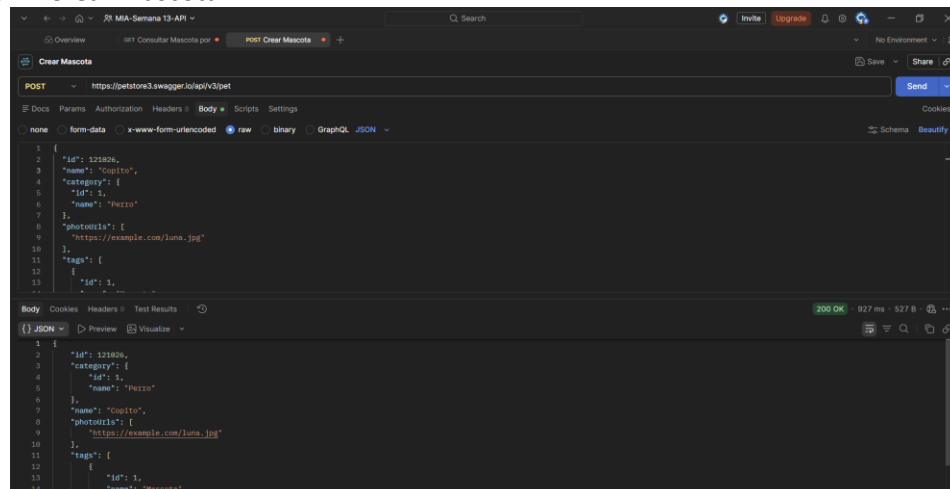


6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

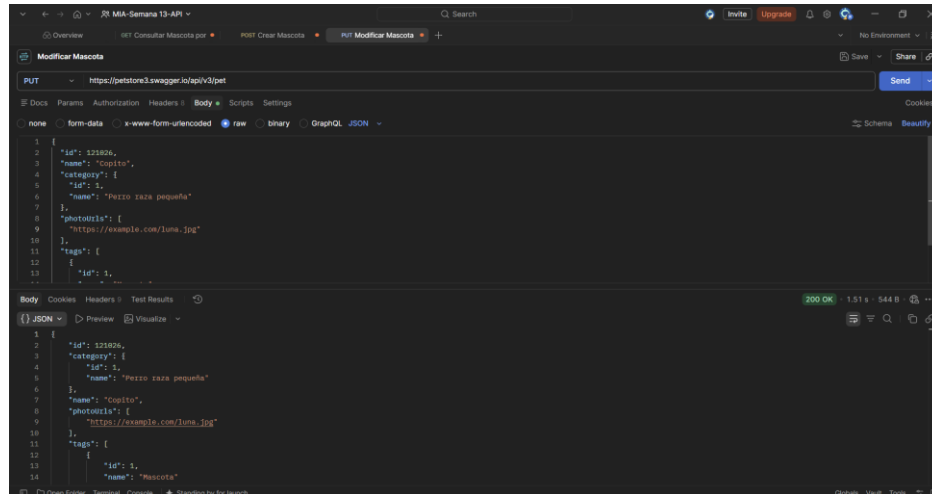


- POST – Crear mascota

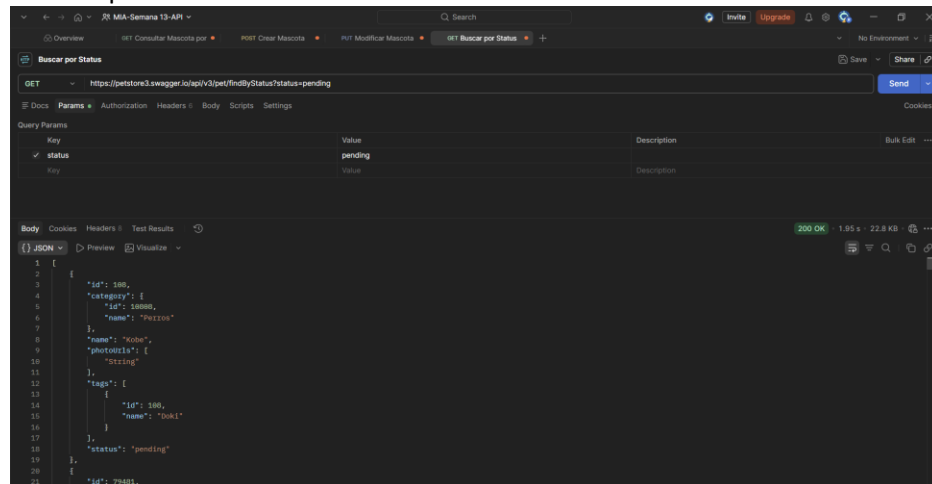




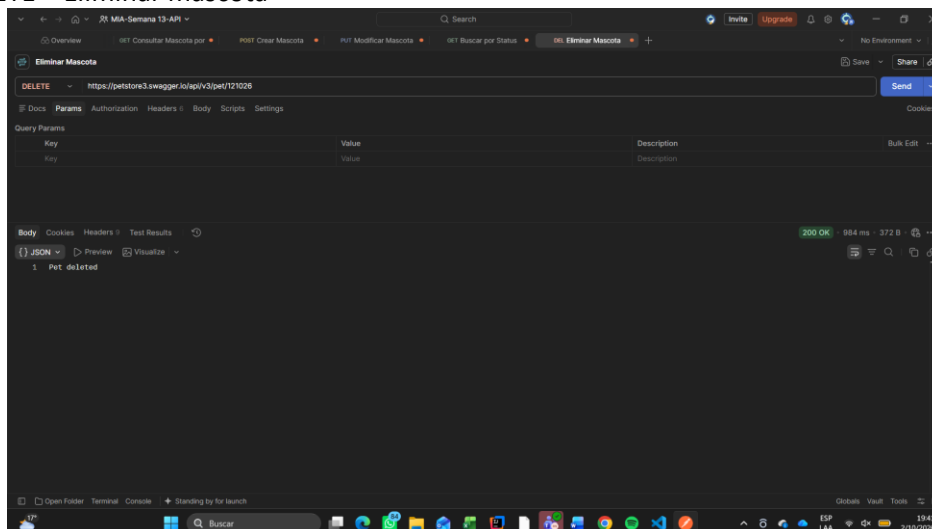
- PUT – Modificar mascota



- GET – Buscar por status

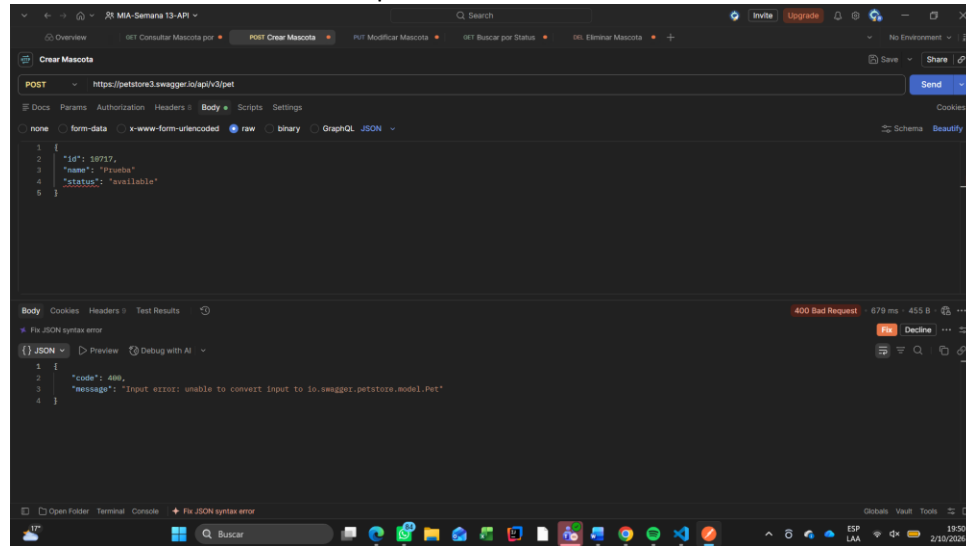


- DELETE – Eliminar mascota



- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

Esto ocurrió porque se realizó la consulta utilizando un ID que no corresponde a ninguna mascota registrada en ese momento en la API.



Solicitud realizada:

Se realizó una solicitud POST `/pet` enviando en el body un JSON con una estructura incorrecta.

Código HTTP recibido:

400 Bad Request.

Respuesta de la API:

La API indicó que ocurrió un error de entrada porque no fue posible convertir la información enviada al modelo Pet.

¿Qué esperaba que ocurriera?

Se esperaba que la API rechazara la solicitud debido a que el JSON enviado estaba incompleto o tenía una estructura incorrecta.

¿Qué ocurrió realmente?

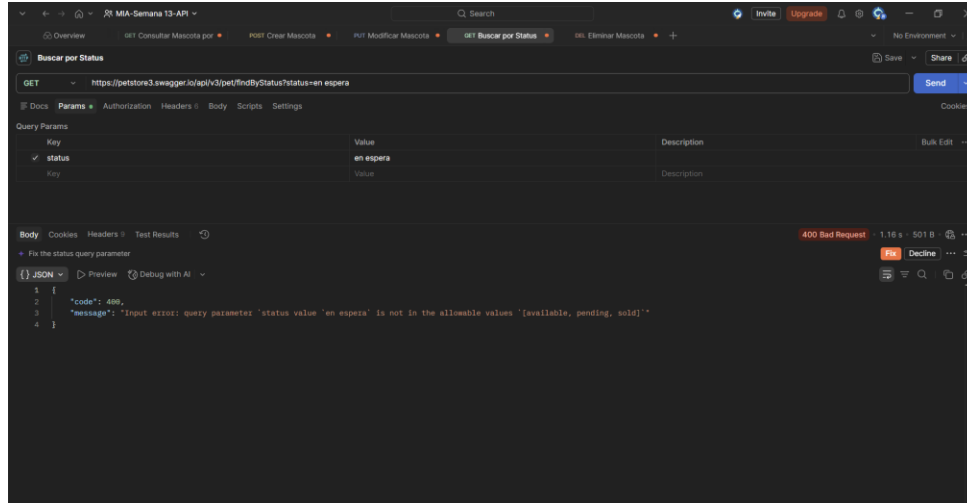
La API devolvió el código 400 Bad Request y un mensaje indicando que no pudo convertir la información recibida al modelo Pet.

Explicación de la causa:

El error ocurrió porque los datos enviados en el cuerpo de la solicitud no cumplían con la estructura esperada por la API. Por esta razón, no fue posible interpretar correctamente la información de la mascota.



3. Utilizar un parámetro con un valor no esperado.



Solicitud realizada:

Se realizó una solicitud GET `/pet/findByStatus` utilizando el parámetro `status` con el valor `en espera`.

Código HTTP recibido:

400 Bad Request.

Respuesta de la API:

La API indicó que el valor "en espera" no se encuentra dentro de los valores permitidos para `status`, los cuales son `available`, `pending` y `sold`.

¿Qué esperaba que ocurriera?

Se esperaba que la API rechazara la solicitud porque se utilizó intencionalmente un valor no válido para el parámetro.

¿Qué ocurrió realmente?

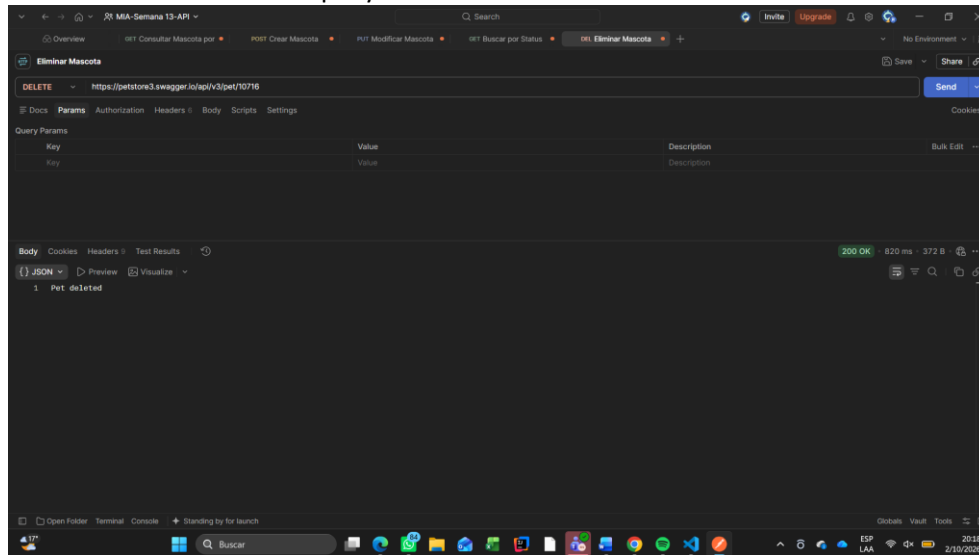
La API rechazó la solicitud y devolvió 400 Bad Request, indicando que el valor utilizado no estaba permitido.

Explicación:

El error ocurrió porque `status` solamente acepta los valores `available`, `pending` y `sold`. El valor "en espera" no forma parte de las opciones admitidas por la API.



4. Intentar eliminar un recurso que ya no existe.



Solicitud realizada:

Se realizó nuevamente una solicitud DELETE sobre la mascota con ID 10716, después de haberla eliminado previamente.

Código HTTP recibido:

200 OK.

Respuesta de la API:

La API respondió con el mensaje:

Pet deleted

¿Qué esperaba que ocurriera?

Se esperaba que la API indicara que la mascota ya no existía o que devolviera un código de error, debido a que el recurso había sido eliminado anteriormente.

¿Qué ocurrió realmente?

La API volvió a responder con 200 OK y el mensaje Pet deleted, como si la eliminación se hubiera realizado correctamente otra vez.

Explicación de la causa:

En esta prueba, Petstore no produjo el error esperado al intentar eliminar nuevamente el mismo recurso. La API de pruebas respondió exitosamente a la operación DELETE aunque el recurso ya había sido eliminado. Por ello, el resultado real fue diferente al comportamiento que se esperaba comprobar.



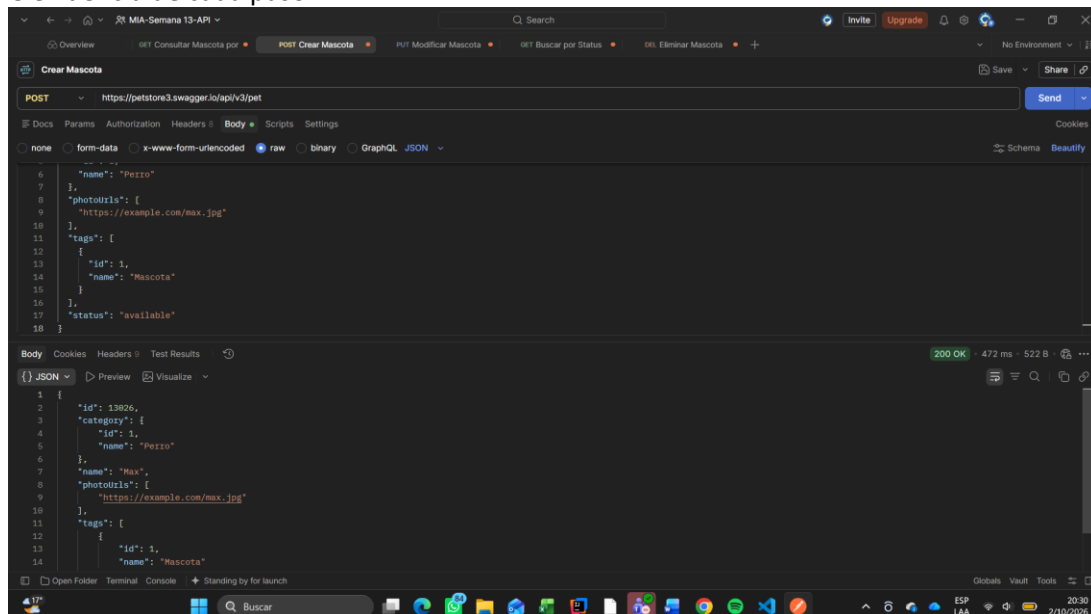
Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
 - **Para registrar una mascota se utiliza /pet; para consultar una mascota por su ID, /pet/{petId}; para actualizar sus datos, /pet; y para eliminarla, /pet/{petId}.**
- Defina el método HTTP correspondiente.
 - **POST para registrar una nueva mascota.**
 - **GET para consultar la información de una mascota.**
 - **PUT para actualizar su información o estado.**
 - **DELETE para eliminarla.**
- Indique qué respuesta debe analizar el programa.
 - **El programa debe analizar principalmente el código de estado HTTP y el cuerpo de la respuesta (Response Body). De esta manera puede determinar si la operación fue exitosa o si ocurrió algún error, por ejemplo, si la mascota solicitada no existe.**
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.





Overview GET Consultar Mascota por ID POST Crear Mascota PUT Modificar Mascota GET Buscar por Status DELETE Eliminar Mascota

GET https://petstore3.swagger.io/api/v3/pet/13026

Docs Params Authorization Headers Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers Test Results

JSON Preview Visualize

```
1 {
2   "id": 13026,
3   "category": {
4     "id": 1,
5     "name": "Perro"
6   },
7   "name": "Max",
8   "photoUrls": [
9     "https://example.com/max.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "available"
18 }
```

200 OK · 107 ms · 522 B

Open Folder Terminal Console Standing by for launch

Global Vault Tools

ESP LAA 20:31 2/10/2026

Overview GET Consultar Mascota por ID POST Crear Mascota PUT Modificar Mascota GET Buscar por Status DELETE Eliminar Mascota

PUT https://petstore3.swagger.io/api/v3/pet

Docs Params Authorization Headers Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
6   "name": "Perro"
7 },
8   "photoUrls": [
9     "https://example.com/max.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "pending"
18 }
```

Body Cookies Headers Test Results

JSON Preview Visualize

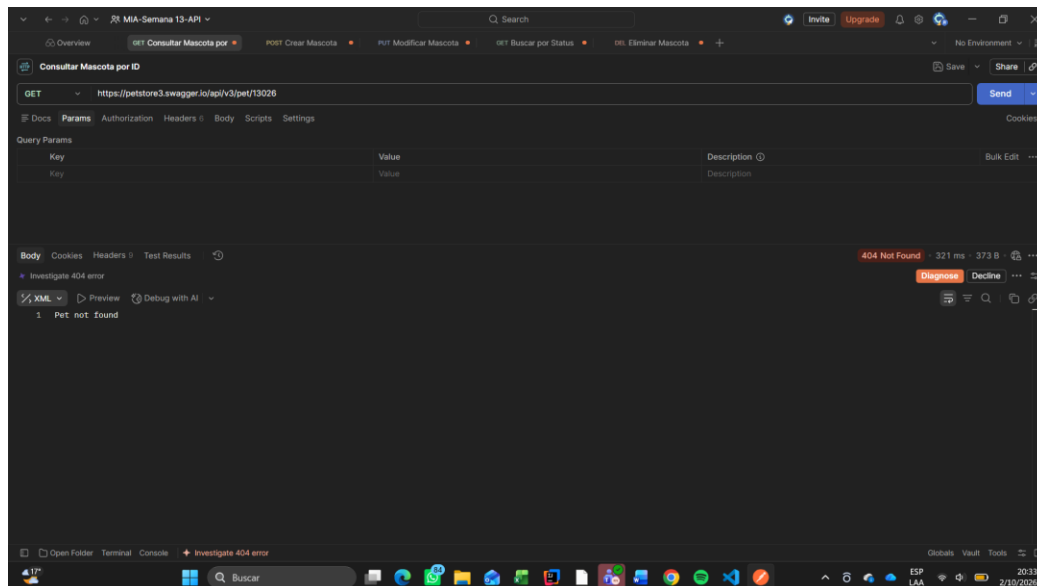
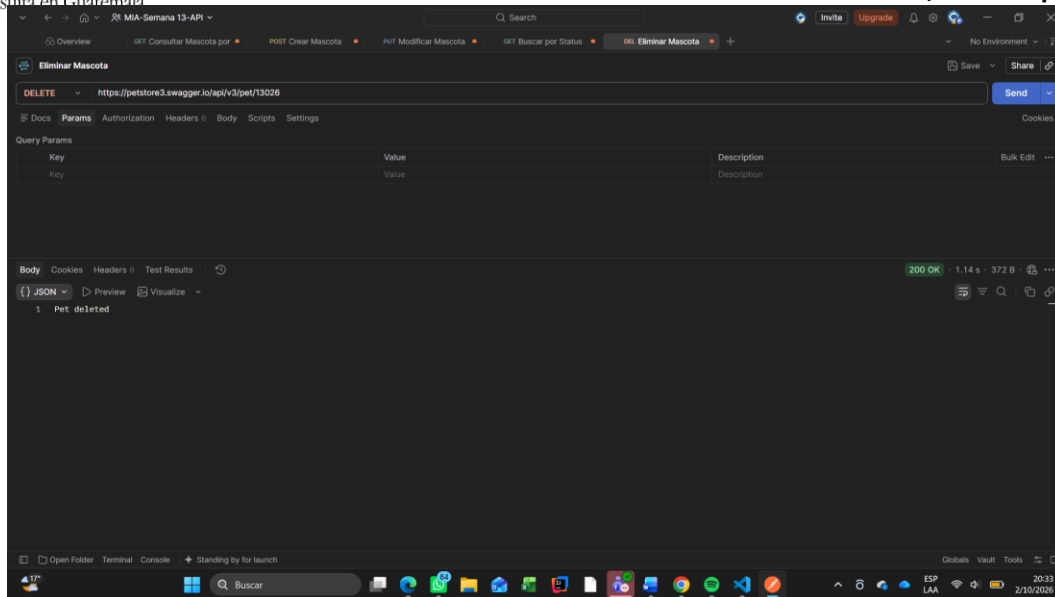
```
1 {
2   "id": 13026,
3   "category": {
4     "id": 1,
5     "name": "Perro"
6   },
7   "name": "Max",
8   "photoUrls": [
9     "https://example.com/max.jpg"
10  ],
11  "tags": [
12    {
13      "id": 1,
14      "name": "Mascota"
15    }
16  ],
17  "status": "pending"
18 }
```

200 OK · 106 ms · 520 B

Open Folder Terminal Console Standing by for launch

Global Vault Tools

ESP LAA 20:32 2/10/2026



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.